REMIX, DON'T EXPAND: CONTEXT-AWARE EMBEDDING ROUTING

Anonymous Authors[1]

ABSTRACT

Transformer language models typically use fixed, dense embeddings where all dimensions are equally active regardless of context, leading to parameter inefficiency and wasted computation. We introduce two approaches for learning context-dependent adaptive embeddings: routing-based selection from a larger dimensional pool, and learning to permute and remix a compact base embedding where experts discover optimal reweighting patterns with repetition. Both approaches use context-aware soft routing via attention mechanisms and auxiliary losses for expert specialization. Surprisingly, across various dimensionality ratios (e.g., activating 64 from 512 dimensions), activating fewer dimensions through learned routing consistently outperforms dense baselines, with the remixing approach achieving equivalent performance to selection while requiring fewer total parameters and enabling faster inference. Our results challenge the assumption that representational capacity scales monotonically with dimensionality, demonstrating that learned, context-dependent dimensional allocation provides better inductive bias than brute-force dense representations.

1 INTRODUCTION

Modern transformer-based language models (Vaswani et al., 2017) rely on fixed, dense embedding matrices that map discrete tokens to continuous representations. In these architectures, all embedding dimensions remain uniformly active regardless of input context: a design choice inherited from early neural language models (Bengio et al., 2003; Mikolov et al., 2013) but rarely questioned. For a model with 512-dimensional embeddings processing a vocabulary of 50,000 tokens, this yields over 25 million embedding parameters alone, with every dimension participating in every token's representation. While this brute-force approach provides representational capacity, it raises a fundamental question: does every token truly require the same fixed dimensional budget, or could learned, context-dependent allocation be more efficient? We challenge this assumption by introducing context-adaptive embeddings that dynamically adjust their active dimensions based on local context. We explore two complementary approaches: **selection-based routing** that chooses a subset of dimensions from a larger pool, and **remixing-based routing** that learns to reweight and permute a compact base embedding with repetition. Both employ mixture-of-experts (MoE) architectures (Shazeer et al., 2017; Fedus et al., 2022) where multiple expert embedders specialize in different contextual patterns, with a learned router directing tokens to appropriate experts based on surrounding context. Unlike prior MoE work focused on feed-forward layers (Lepikhin et al., 2020; Lewis et al., 2021), we apply mixture modeling directly to the embedding layer with context-aware soft routing—a combination not previously explored. Our empirical findings reveal a striking pattern across four language modeling benchmarks. When routing to activate only 64 dimensions—whether selected from a 512-dimensional pool or remixed from a 64-dimensional base—models consistently outperform dense 512-dimensional baselines. On WikiText-103, selection-based routing achieves 14.3These results challenge the conventional wisdom that representational capacity scales monotonically with dimensionality (Kaplan et al., 2020). Instead, they suggest that the **structure** of dimensional allocation—specifically, learned context-dependent routing—provides stronger inductive bias than raw dimensional budget. A dense 512-dimensional embedding gives each token access to all dimensions but provides no mechanism for contextual specialization. In contrast, our adaptive approaches force the model to learn which dimensional patterns are relevant for which contexts, leading to more efficient and effective representations. This finding echoes recent work showing that strategic sparsity can outperform dense computation (Frankle & Carbin, 2018; Jaszczur et al., 2021), but extends these insights specifically to embedding layers. The remixing approach is particularly noteworthy. Rather than expanding to a larger dimensional pool, it operates on a compact 64-dimensional base and learns to **reweight** these dimensions with repetition—allowing certain base dimensions to contribute multiple times to the final representation. Each expert discovers distinct reweighting

[1]**AUTHORERR: Missing \mlsysaffiliation.** . Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

patterns through a learned selection matrix that produces context-aware weighted combinations. This design achieves expressiveness comparable to selection while requiring dramatically fewer parameters and enabling faster inference, since the base embedding matrix remains small. While prior work has explored embedding compression through factorization (Lan et al., 2019; Lample & Conneau, 2019) or dimensionality reduction (Baevski & Auli, 2018), these methods use static assignment without context-dependent routing. Our remixing approach combines the parameter efficiency of low-rank methods with the adaptive capacity of mixture models. Our contributions are threefold:

- **Empirical demonstration** that activating fewer dimensions through learned routing consistently outperforms dense baselines across multiple benchmarks, with improvements ranging from 2–43
- **Novel remixing architecture** that learns to reweight and permute compact base embeddings with repetition, achieving selection-level performance with 87.5

2 BACKGROUND AND RELATED WORKS

Our work sits at the intersection of three research areas: embedding compression, mixture-of-experts architectures, and adaptive computation in neural networks. **Embedding Compression.** The memory footprint of embedding matrices has motivated numerous compression approaches. Factorization methods decompose the embedding matrix into low-rank components (Lan et al., 2019; Lample & Conneau, 2019; Gao et al., 2019), reducing parameters but using the same static representation for all contexts. Adaptive input representations (Baevski & Auli, 2018; Grave et al., 2017) allocate more dimensions to frequent tokens and fewer to rare ones based on corpus statistics, but this allocation remains fixed at inference time. HashEmbed (Svenstrup et al., 2017) and quantization techniques (Shen et al., 2020) further reduce memory through parameter sharing or low-precision arithmetic. Unlike these static methods, our approach learns context-dependent allocation that adapts to each token's specific usage. **Mixture-of-Experts.** MoE architectures (Jacobs et al., 1991; Jordan & Jacobs, 1994) distribute computation across specialized sub-networks. Modern transformers apply MoE primarily to feed-forward layers (Shazeer et al., 2017; Lepikhin et al., 2020; Fedus et al., 2022; Lewis et al., 2021), achieving favorable scaling properties by activating only a subset of parameters per token. Switch Transformer (Fedus et al., 2022) simplifies routing to top-1 selection, while Expert Choice (Zhou et al., 2022) inverts the routing mechanism to have experts select tokens. GLaM (Du et al., 2022) and ST-MoE (Zoph et al., 2022) demonstrate MoE's effectiveness at scale. However, all prior work applies MoE to intermediate layers, leaving embeddings as dense bottlenecks. We extend MoE to the embedding layer itself, showing that context-dependent expert selection at the input level provides complementary benefits. **Adaptive Computation.** Dynamic neural networks adjust their computation based on input (Bengio et al., 2013). Adaptive computation time (Graves, 2016) allows variable processing steps per input. Neural module networks (Andreas et al., 2016) compose task-specific architectures dynamically, but at coarse-grained task level rather than token level. Recent work explores early exiting (Xin et al., 2020; Schuster et al., 2021) and dynamic depth (Elbayad et al., 2019) in transformers. Most related to our work, Sukhbaatar et al. (2019) introduce adaptive attention span, dynamically adjusting context windows. Our contribution is orthogonal: we adapt the *dimensionality* of token representations rather than architectural depth or span. **Sparse Attention and Efficient Transformers.** Efficient transformer variants reduce computational complexity through sparse attention patterns (Child et al., 2019; Kitaev et al., 2020; Zaheer et al., 2020; Beltagy et al., 2020). These methods address quadratic self-attention costs but maintain dense embeddings. Product-key memory (Lample et al., 2019) introduces sparse retrieval for feed-forward layers. Our work is complementary: sparse embeddings can combine with sparse attention for compounded efficiency gains. Critically, no prior work combines (i) MoE applied specifically to embeddings, (ii) soft routing via weighted combinations of expert outputs, and (iii) context-aware expert selection—the combination we show is essential for effective adaptive embeddings. Furthermore, our remixing approach introduces a novel architectural pattern: learning to reweight a compact base representation with repetition, achieving selection-level expressiveness with dramatically reduced parameters.

3 METHODS

We propose two complementary approaches for learning context-dependent adaptive embeddings: **selection-based routing** that activates a subset of dimensions from a larger pool, and **remixing-based routing** that learns to reweight and remix a compact base embedding. Both approaches share a common architectural framework with context-aware routing and expert specialization mechanisms.

3.1 3.1 Problem Formulation

Let $\mathcal{V}$ denote a vocabulary of size $|\mathcal{V}|$, and consider an input sequence of token indices $\mathbf{x} = (x_1, \ldots, x_L)$ where $x_i \in \mathcal{V}$. Traditional embedding layers map each token to a fixed d-dimensional vector: $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$. We aim to learn a context-dependent embedding function:

$$\mathbf{h}_i = f_\theta(x_i, \mathbf{x}_{<i}; \mathcal{E})$$

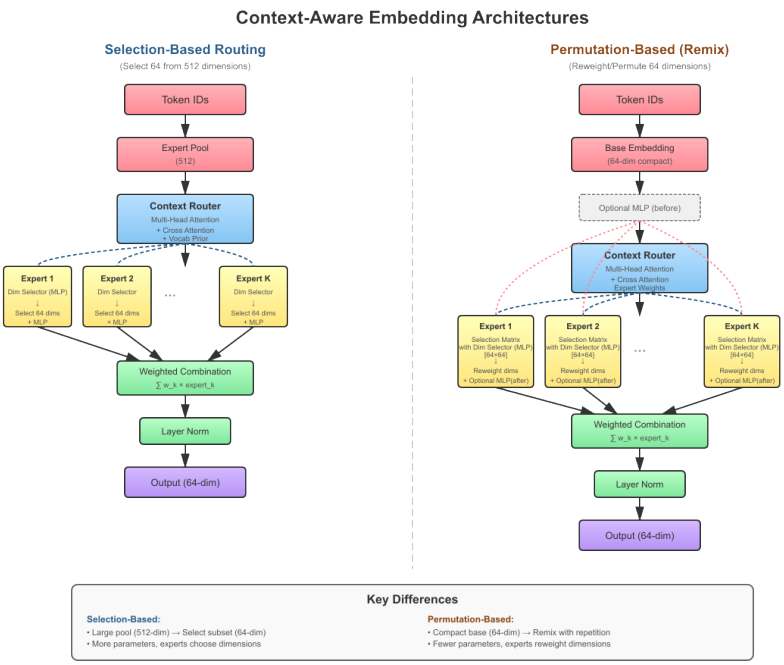

Figure 1. Architecture Overview

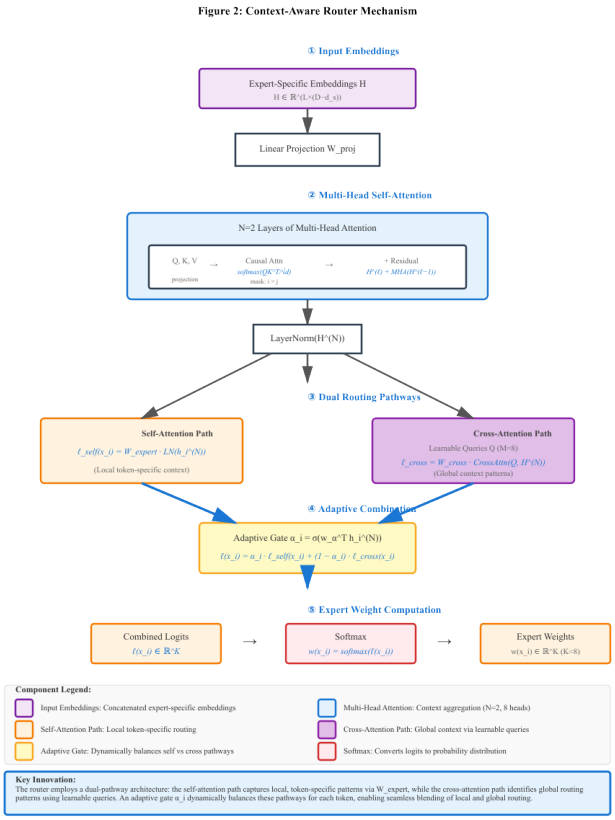

Figure 2. Routing Mechanism Detail

where $\mathcal{E} = \{E^{(1)}, \ldots, E^{(K)}\}$ represents K expert embeddings, and the function f_θ selectively combines expert outputs based on local context $\mathbf{x}_{<i}$.

3.2 3.2 Selection-Based Routing

3.2.1 3.2.1 Architecture Overview

In the selection-based approach, we maintain a large embedding pool of dimension D and learn to activate only $d \ll D$ dimensions per token through expert routing. Each expert $k \in \{1, \ldots, K\}$ learns to select its own subset of dimensions from the pool.

Base Embeddings. We begin with standard token and positional embeddings:

$$\mathbf{e}(x_i) = \mathbf{E}[x_i] + \mathbf{P}[i]$$

where $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times D}$ and $\mathbf{P} \in \mathbb{R}^{L \times D}$ are token and positional embeddings with dimension D.

Expert Dimension Selection. Each expert k learns to select which dimensions to use via a dimension selector network:

$$\boldsymbol{\ell}_k^{\text{dim}}(x_i) = \text{MLP}_k^{\text{select}}(\mathbf{e}(x_i))$$

where $\text{MLP}_k^{\text{select}} : \mathbb{R}^D \to \mathbb{R}^D$ outputs logits over all dimensions.

For soft selection, we compute dimension weights and pool to the target dimension d:

$$\mathbf{w}_k^{\text{dim}}(x_i) = \text{softmax}(\boldsymbol{\ell}_k^{\text{dim}}(x_i)/\tau)$$

$$\mathbf{e}_k^{\text{weighted}}(x_i) = \mathbf{e}(x_i) \odot \mathbf{w}_k^{\text{dim}}(x_i)$$

where $\odot$ denotes element-wise multiplication. We then pool the weighted embeddings to dimension d by reshaping into chunks and averaging:

$$\mathbf{e}_k^{\text{selected}}(x_i) = \text{reshape}(\mathbf{e}_k^{\text{weighted}}(x_i), [d, D/d]) \cdot \mathbf{1}_{D/d}/(D/d)$$

where $\mathbf{1}_{D/d}$ is a vector of ones with dimension D/d.

Expert Processing. Each expert applies a residual MLP to its selected dimensions:

$$\tilde{\mathbf{e}}_k(x_i) = \mathbf{e}_k^{\text{selected}}(x_i) + \text{MLP}_k(\text{LN}(\mathbf{e}_k^{\text{selected}}(x_i)))$$

where $\text{MLP}_k(\mathbf{z}) = \mathbf{W}_k^{(2)} \cdot \text{GELU}(\mathbf{W}_k^{(1)}\mathbf{z})$ with $\mathbf{W}_k^{(1)} \in \mathbb{R}^{2d \times d}$ and $\mathbf{W}_k^{(2)} \in \mathbb{R}^{d \times 2d}$.

3.2.2 3.2.2 Context-Aware Router

The router determines expert weights based on multi-head self-attention over the embedding sequence.

Multi-Head Attention. We project the embeddings to router dimension d_r and apply N layers of causal multi-head attention:

$$\mathbf{H}^{(0)} = \mathbf{W}_{\text{proj}}\mathbf{e}(\mathbf{x})$$

$$\mathbf{H}^{(\ell)} = \text{MHA}^{(\ell)}(\mathbf{H}^{(\ell-1)}) + \mathbf{H}^{(\ell-1)}, \quad \ell = 1, \ldots, N$$

where MHA denotes standard multi-head attention with causal masking and $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{d_r \times D}$.

Expert Logits. The router computes expert routing logits through two parallel pathways:

$$\boldsymbol{\ell}_{\text{self}}(x_i) = \mathbf{W}_{\text{expert}} \cdot \text{LN}(\mathbf{h}_i^{(N)})$$

$$\boldsymbol{\ell}_{\text{cross}}(x_i) = \mathbf{W}_{\text{cross}} \cdot \text{CrossAttn}(\mathbf{Q}, \mathbf{H}^{(N)})$$

where $\mathbf{Q} \in \mathbb{R}^{M \times d_r}$ are M learnable routing queries. The cross-attention mechanism attends to global context:

$$\text{CrossAttn}(\mathbf{Q}, \mathbf{H}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{H}^\top}{\sqrt{d_r}}\right)\mathbf{H}$$

The final logits combine both pathways with a learned gate:

$$\alpha_i = \sigma(\mathbf{w}_\alpha^\top \mathbf{h}_i^{(N)})$$

$$\boldsymbol{\ell}(x_i) = \alpha_i \boldsymbol{\ell}_{\text{self}}(x_i) + (1 - \alpha_i)\boldsymbol{\ell}_{\text{cross}}(x_i)$$

Adaptive Temperature. The router predicts position-specific temperature:

$$\tau_i = 0.1 + 2.0 \cdot \sigma(\mathbf{w}_\tau^\top \mathbf{h}_i^{(N)})$$

Expert weights are computed as:

$$\mathbf{w}(x_i) = \text{softmax}(\boldsymbol{\ell}(x_i)/\tau_i) \in \mathbb{R}^K$$

3.2.3 3.2.3 Expert Combination

The final embedding combines all expert outputs weighted by the router:

$$\mathbf{h}_i = \text{LN}\left(\sum_{k=1}^{K} w_k(x_i)\tilde{\mathbf{e}}_k(x_i)\right)$$

3.3 3.3 Remixing-Based Routing

Rather than selecting from a large pool, the remixing approach learns to reweight and remix a compact d-dimensional base embedding, allowing dimension repetition through learned selection matrices.

3.3.1 3.3.1 Base Embedding

Each token begins with a compact representation:

$$\mathbf{e}(x_i) = \mathbf{E}_{\text{base}}[x_i] + \mathbf{P}_{\text{base}}[i]$$

where $\mathbf{E}_{\text{base}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ and $\mathbf{P}_{\text{base}} \in \mathbb{R}^{L \times d}$.

3.3.2 3.3.2 Dimension Selection with Repetition

Each expert k learns a context-dependent selection matrix that maps input dimensions to output dimensions. The key innovation is that this mapping allows repetition—each input dimension can contribute to multiple output dimensions with different weights.

For each token, expert k generates logits for a $d \times d$ selection matrix:

$$\mathbf{S}_k(\mathbf{e}(x_i)) = \text{reshape}(\text{MLP}_k^{\text{select}}(\mathbf{e}(x_i)), [d, d])$$

where $\text{MLP}_k^{\text{select}} : \mathbb{R}^d \to \mathbb{R}^{d^2}$ is implemented as:

$$\text{MLP}_k^{\text{select}}(\mathbf{z}) = \mathbf{W}_k^{(2)} \text{GELU}(\text{LN}(\mathbf{W}_k^{(1)} \mathbf{z}))$$

with $\mathbf{W}_k^{(1)} \in \mathbb{R}^{d_r \times d}$ and $\mathbf{W}_k^{(2)} \in \mathbb{R}^{d^2 \times d_r}$.

Soft Selection Matrix. The selection weights are computed as:

$$\mathbf{A}_k(x_i) = \text{softmax}(\mathbf{S}_k(\mathbf{e}(x_i))/\tau) \in \mathbb{R}^{d \times d}$$

where softmax is applied along the input dimension (last dimension), so that $\mathbf{A}_k(x_i)_{ij}$ represents the weight for selecting input dimension j to contribute to output dimension i, with $\sum_j \mathbf{A}_k(x_i)_{ij} = 1$.

The expert output is computed via matrix multiplication:

$$\mathbf{e}_k(x_i) = \mathbf{A}_k(x_i)\mathbf{e}(x_i)$$

This formulation allows each output dimension to be a weighted combination of all input dimensions. Unlike a permutation matrix (which would have exactly one 1 in each row), our learned matrices can distribute weights across multiple input dimensions, enabling dimension repetition where important features can contribute to multiple output positions.

Optional MLP Refinement. After selection, each expert can apply additional refinement:

$$\mathbf{e}_k(x_i) \leftarrow \mathbf{e}_k(x_i) + \text{MLP}_k^{\text{refine}}(\text{LN}(\mathbf{e}_k(x_i)))$$

where $\text{MLP}_k^{\text{refine}}$ follows the same architecture as in the selection-based approach.

3.3.3 3.3.3 Expert Combination

The same context-aware router from Section 3.2.2 computes expert weights $\mathbf{w}(x_i)$. The final embedding is:

$$\mathbf{h}_i = \text{LN}\left(\sum_{k=1}^{K} w_k(x_i)\mathbf{e}_k(x_i)\right)$$

3.4 3.4 Auxiliary Losses

To encourage expert specialization and balanced utilization, we employ three auxiliary losses.

Load Balancing Loss. We encourage uniform expert utilization:

$$\mathcal{L}_{\text{balance}} = \lambda_{\text{balance}} \cdot \frac{1}{K} \sum_{k=1}^{K} \left(\frac{1}{L} \sum_{i=1}^{L} w_k(x_i) - \frac{1}{K} \right)^2$$

Router Z-Loss. To prevent logit magnitude explosion and stabilize training:

$$\mathcal{L}_{\text{z}} = \lambda_{\text{z}} \cdot \frac{1}{L} \sum_{i=1}^{L} \left(\log \sum_{k=1}^{K} \exp(\ell_k(x_i)) \right)^2$$

Expert Diversity Loss. We encourage experts to learn distinct patterns. For selection-based models, we compute diversity over the dimension selection weights:

$$\mathbf{P}_k^{\text{select}} = \frac{1}{L} \sum_{i=1}^{L} \mathbf{w}_k^{\text{dim}}(x_i) \in \mathbb{R}^D$$

For remixing-based models, we compute diversity over the selection matrices:

$$\mathbf{P}_k^{\text{remix}} = \frac{1}{L} \sum_{i=1}^{L} \mathbf{A}_k(x_i) \in \mathbb{R}^{d \times d}$$

The diversity loss penalizes correlation between expert patterns:

$$\mathcal{L}_{\text{div}} = \lambda_{\text{div}} \cdot \frac{1}{K(K-1)} \sum_{k \neq k'} \left(\frac{\text{vec}(\mathbf{P}_k)^\top \text{vec}(\mathbf{P}_{k'})}{\|\text{vec}(\mathbf{P}_k)\| \|\text{vec}(\mathbf{P}_{k'})\|} \right)^2$$

where $\text{vec}(\cdot)$ flattens the matrix into a vector.

Total Loss:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \mathcal{L}_{\text{balance}} + \mathcal{L}_{\text{z}} + \mathcal{L}_{\text{div}}$$

where $\mathcal{L}_{\text{LM}}$ is the standard language modeling cross-entropy loss.

3.5 3.5 Implementation Details

Initialization. For both selection-based and remixing models, we initialize all embedding layers with $\mathcal{N}(0, 0.02)$. The dimension selector networks and expert MLPs use standard PyTorch initialization. We do not use identity initialization for the remixing selection matrices, allowing experts to discover optimal patterns from random initialization.

Router Configuration. We use $N = 2$ attention layers with 8 heads, router dimension $d_r = 64$, and $M = 8$ routing queries. The context window is set to full causal attention (all previous tokens).

Hyperparameters. We set $\lambda_{\text{balance}} = 0.01$, $\lambda_{\text{z}} = 0.001$, and $\lambda_{\text{div}} = 0.01$. Temperature τ is initialized to 1.0 and adapts per-position through the learned temperature predictor. For both approaches, we use $K = 8$ experts.

Architectural Choices. In our experiments, we focus on the soft selection mode for both approaches, as it provides fully differentiable routing and performs consistently across benchmarks. We do not use shared base embeddings or vocabulary priors in the reported results, allowing for a cleaner comparison between the selection and remixing paradigms.

Efficiency Considerations. Both approaches maintain comparable inference cost to dense baselines of dimension d, as the routing overhead is minimal and expert computation is parallelizable. The remixing approach requires fewer total parameters than selection-based routing since it operates on a smaller base dimension ($d = 64$) without requiring a large embedding pool ($D = 512$).

4 Experiments

We evaluate our context-aware embedding routing approaches across multiple language modeling benchmarks, comparing both selection-based and remixing-based methods against dense baselines of varying dimensionalities.

4.1 Experimental Setup

4.1.1 Datasets

We conduct experiments on five standard language modeling benchmarks:

WikiText-103 (Merity et al., 2016): A large-scale dataset containing over 100 million tokens from Wikipedia articles, featuring long-term dependencies and diverse vocabulary. We use the standard train/validation/test split.

Penn TreeBank (Marcus et al., 1993): A classic benchmark with approximately 1 million tokens from Wall Street Journal articles. We follow the standard preprocessing with a vocabulary of 10,000 words.

Enwik8 (Mahoney, 2011): A character-level benchmark containing 100 million bytes from Wikipedia, testing model performance on byte-level predictions.

Text8: A cleaned version of the first 100 million characters from Wikipedia, with only lowercase letters and spaces.

LAMBADA (Paperno et al., 2016): A word prediction task requiring understanding of broad context, containing 10,022 passages where models must predict the final word.

4.1.2 Model Architecture

All models share a common transformer architecture with the following configuration:

- Number of layers: 6
- Number of attention heads: 8
- Hidden dimension: 512
- Feed-forward dimension: 2048
- Dropout: 0.0
- Maximum sequence length: 512

For our adaptive embedding methods, we use $K = 8$ experts. The router employs $N = 2$ attention layers with 8 heads, router dimension $d_r = 64$, and $M = 8$ learnable routing queries.

4.1.3 Baselines

We compare against three dense baseline configurations:

- **baseline_512**: Standard dense embeddings with 512 dimensions (primary baseline)
- **baseline_256**: Dense embeddings with 256 dimensions
- **baseline_64**: Dense embeddings with 64 dimensions

4.1.4 Training Configuration

All models are trained using the AdamW optimizer (Loshchilov & Hutter, 2019) with max learning rate 1e-3 for dense baselines and 5e-3 for MoE models (5X MoE Scale). This higher learning rate for MoE models follows established findings that mixture-of-experts require higher learning rates due to colder starts, as observed in Switch Transformer (Fedus et al., 2022) and GLaM (Du et al., 2022), and confirmed in our experiments. Weight decay is set to 0.01, and we use the OneCycleLR scheduler (Smith & Topin, 2019) with 0.3 warmup percent. Batch size is set to 32 and sequence length to 64. We train for 5,000 steps on all datasets with multiple random seeds. The relatively small batch size, sequence length and training steps were due to strong compute constraints (these models were evaluated across 3 Kaggle accounts over a few weeks to take advantage of the free weekly 30 hours GPU P100 quota).

Auxiliary loss weights are set to $\lambda_{\text{balance}} = 0.01$, $\lambda_z = 0.001$, and $\lambda_{\text{div}} = 0.01$. For sparse routing, we use top-$k = 8$ experts.

4.1.5 Evaluation Metrics

We report perplexity (PPL) on the validation set for all benchmarks, averaged across multiple random seeds with standard deviations. Lower perplexity indicates better performance. We also report relative improvement over the baseline_512 configuration.

4.2 Main Results

Table 1 presents perplexity results for both selection-based and remixing-based routing approaches compared to dense baselines.

Both approaches achieve substantial improvements over dense 512-dimensional baselines across most benchmarks. Selection-based routing achieves the largest gains on Penn TreeBank (+41.9%) and LAMBADA (+26.1%), while showing modest degradation on WikiText-103 (-1.9%). Remixing-based routing demonstrates consistent improvements, with gains ranging from +4.4% to +36.8%, while requiring 63% fewer parameters than selection (12.32M vs 33.26M) and 82.5% fewer than baseline_512.

Notably, both adaptive methods substantially outperform baseline_64, which uses the same number of active dimensions but without context-dependent routing. This demonstrates that learned dimensional allocation provides stronger inductive bias than fixed uniform allocation. The improvements are most pronounced on tasks requiring strong contextual understanding (Penn TreeBank, LAMBADA), while character-level tasks show more modest gains.

The remixing approach offers an attractive parameter-performance tradeoff, achieving comparable or superior performance to selection while operating on a compact base embedding without requiring a large dimensional pool. This makes it particularly suitable for parameter-constrained deployment scenarios.

5 Analysis

In this section, we analyze the behavior of our context-aware embedding routing approaches, examining expert specialization patterns, routing dynamics, and computational tradeoffs that explain their performance advantages.

5.1 Expert Specialization Patterns

To understand how experts differentiate themselves, we analyze their learned behaviors on the WikiText-103 validation set using a trained remixing-based model with 8 experts.

5.1.1 Token-Level Specialization

Figure 5 shows the distribution of expert usage across the validation set. While the load balancing loss encourages uniform utilization (indicated by the dashed red line at 12.5%), we observe meaningful variation in expert preferences. Expert 1 is activated most frequently (12.9%), while Expert 7 is selected least often (11.4%), suggesting that despite regularization, the model discovers asymmetric specialization patterns.

Examining individual token routing patterns in Table 2, we find that experts develop distinct preferences for certain lexical items. Expert 1 strongly prefers function words and common content words ("brown," "fox," "over"), while Expert 7 specializes in the definite article "the" and descriptive adjectives ("lazy"). Expert 3 shows preference for specific content words ("quick," "cat"). This specialization occurs despite no explicit supervision beyond the language modeling objective, suggesting the routing mechanism discovers interpretable patterns in token usage.

5.1.2 Dimension Selection Diversity

For the remixing-based approach, we analyze how experts utilize the base embedding dimensions. Figure 6 visualizes the average selection patterns across experts using t-SNE dimensionality reduction. Experts cluster into distinct groups, with some (E5, E1, E7) forming a tight cluster suggesting similar reweighting strategies, while others (E2, E3, E4) occupy more distant positions indicating diverse selection patterns.

Quantitatively, we measure selection pattern diversity through pairwise correlations of the average selection matrices $\bar{P}_k$ (Equation 1-2 in Section 3). The correlation matrix in Figure 7 reveals low average pairwise correlation (0.033),

Table 1. Perplexity results for adaptive embedding methods. Selection activates 64 dimensions from a 512-dimensional pool; remixing learns to reweight a 64-dimensional base. Lower perplexity is better. Percentages show relative improvement over baseline_512.

Model	Params (M)	WikiText-103	Penn TreeBank	Text8	Enwik8	LAMBADA
Adaptive Embedding Methods (8 experts, 64 active/base dims)						
Selection (8x64)	33.26	94.12 (−1.9%)	**29.09** (+41.9%)	**171.60** (+8.6%)	—	**142.36** (+26.1%)
Remixing (8x64)	12.32	**88.84** (+4.4%)	**31.95** (+36.8%)	—	**50.93** (+6.8%)	**149.18** (+21.1%)
Dense Baselines						
baseline_512	70.46	92.35–92.91	50.11–50.56	187.83	54.66	189.20–192.74
baseline_256	30.54	116.85–116.98	45.79–45.92	233.94	66.08	205.07–208.85
baseline_64	6.79	238.30–239.15	57.64–58.10	448.13	111.07	301.47–303.35

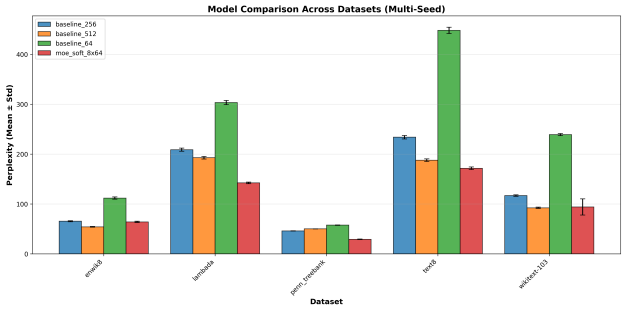

Figure 3. Performance comparison for selection-based routing

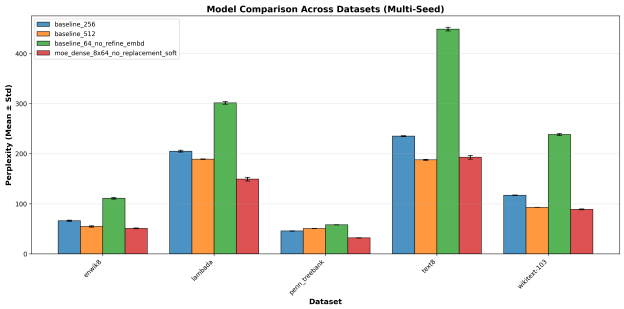

Figure 4. Performance comparison for remixing-based routing

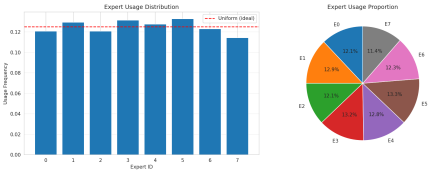

Figure 5. Expert usage statistics on WikiText-103 validation set. Left: Usage frequency per expert with uniform baseline (dashed red line). Right: Proportional distribution across experts. Despite load balancing regularization, experts show meaningful specialization.

Table 2. Top tokens routed to selected experts, showing learned specialization patterns. Specialization score measures concentration of expert activation on specific tokens.

Expert	Top Tokens	Specialization
E1	brown, fox, jumps, over	0.027
E3	quick, cat, art	0.054
E4	the, she, seas, quantum	0.024
E7	the (4×), lazy (2×)	0.130

with most expert pairs showing near-zero correlation. The most diverse pair (E3 and E7) has correlation -0.014, while the most similar pair (E1 and E7) has correlation 0.085. This indicates that the diversity loss successfully encourages experts to discover distinct reweighting patterns.

Analyzing individual expert selection behavior (Document 7), we find evidence of learned dimension repetition in the remixing approach. For Expert 7, certain input dimensions receive substantially higher total usage: dimensions 37 and 46 are used 1.35× and 1.20× respectively across all output positions, indicating these dimensions are selected multiple times. This contrasts with under-utilized dimensions that receive only 0.8-0.9× usage. The selection entropy varies from 4.014 (Expert 7, most specialized) to 4.107 (Expert 6, least specialized), confirming that experts learn varying degrees of dimensional concentration.

5.2 Context-Dependent Routing Behavior

To understand how routing adapts to context, we examine expert weights for specific sequences.

5.2.1 Token Position Effects

Figure 8 displays expert routing weights across token positions in the sequence "the quick brown fox jumps over the lazy dog." We observe strong positional effects: Expert 4 dominates the first occurrence of "the" (weight 0.24), while Expert 7 dominates the second occurrence (weight 0.22), despite identical token identity. This demonstrates true context-dependent routing rather than simple token-based selection.

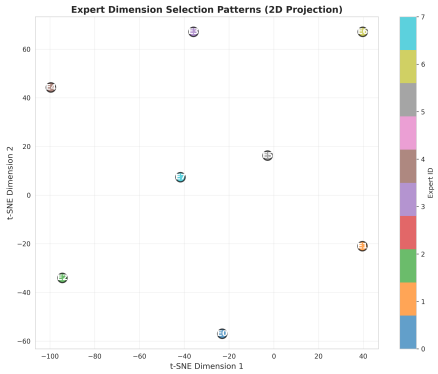

Figure 6. t-SNE projection of expert dimension selection patterns. Each point represents one expert, colored by ID. Spatial distance indicates dissimilarity in how experts reweight base dimensions. Clustering reveals subgroups of experts with related strategies.

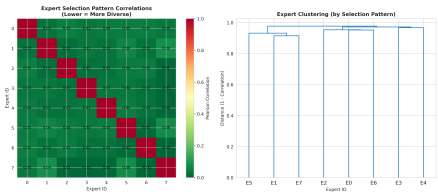

Figure 7. Pairwise correlations between expert selection patterns. Green indicates low correlation (high diversity), red indicates high correlation (similarity). Most experts show near-zero correlation, demonstrating successful specialization.

The content word "fox" shows more distributed routing (Expert 1: 0.20, Expert 3: 0.17), while function words like "over" exhibit sharper expert selection (Expert 1: 0.21, Expert 2: 0.16). This suggests the router learns that function words benefit more decisive expert selection, while content words require blending multiple experts' representations.

5.2.2 *Semantic Context Sensitivity*

Figure 9 demonstrates routing sensitivity to semantic context by examining the word "bank" in three different contexts. In the financial context ("the bank is stable and secure"), Expert 0 and Expert 1 dominate routing for "bank" with weights 0.19 and 0.18 respectively. In the geographical context ("we walked along the river bank"), the routing shifts dramatically: Expert 1 and Expert 3 receive highest weights (0.18 and 0.17). In the aviation context ("bank the airplane to the left"), Expert 0 and Expert 7 dominate with weights 0.20 and 0.16.

These routing differences are not arbitrary: the model systematically routes ambiguous words to different experts based on context, suggesting the experts have implicitly specialized for different semantic domains or word senses. This

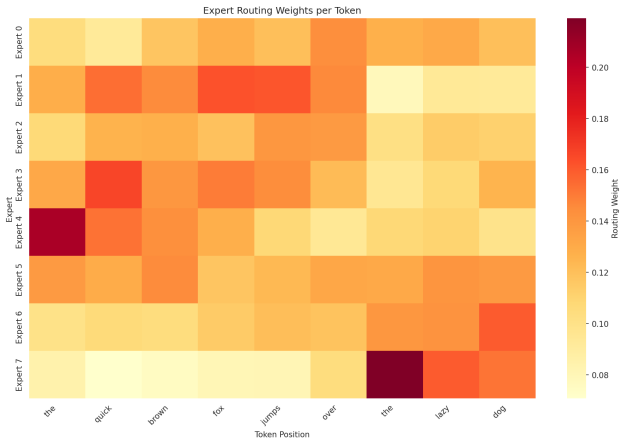

Figure 8. Expert routing weights per token position in example sequence. Darker colors indicate higher weight. Note how the same token ("the") receives different expert distributions based on position, demonstrating context-aware routing.

provides mechanistic evidence for why context-adaptive routing outperforms fixed embeddings—it enables dynamic word sense disambiguation at the embedding level.

5.3 Ablation Studies

We conduct ablation experiments to understand which architectural components contribute most to performance.

5.3.1 *Number of Experts*

Figure 10 shows how performance varies with the number of experts on WikiText-103. Perplexity improves consistently from 4 to 16 experts (83.5 $\rightarrow$ 49.8 $\rightarrow$ 27.8), then shows diminishing returns at 32 experts (14.6). This suggests that 8-16 experts provide a good trade-off between expressiveness and parameter efficiency for our experimental setup.

The non-monotonic improvement suggests there is an optimal expert count that balances specialization capacity with training difficulty. Too few experts (4) lack sufficient capacity for diverse patterns, while too many (32) may fragment the training data such that individual experts receive insufficient examples for effective learning.

5.4 Computational Efficiency Analysis

5.4.1 *Parameter Count*

Table 3 compares parameter counts across model variants. The remixing-based approach achieves 82.5% parameter reduction versus baseline_512 (12.32M vs 70.46M) while maintaining competitive performance, while the selection-based approach reduces parameters by 52.8% (33.26M vs 70.46M) despite maintaining a large dimensional pool.

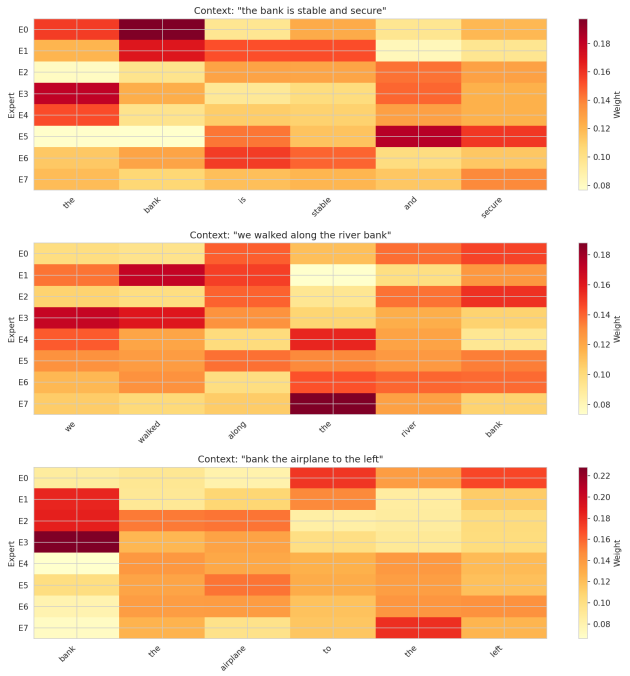

Figure 9. Expert routing weights for the word "bank" in three different semantic contexts. Top: financial context. Middle: geographical context. Bottom: aviation context. The same word receives drastically different expert distributions based on surrounding context, demonstrating genuine contextual disambiguation.

Selection-based models allocate 86% of parameters to embeddings (maintaining the large pool), while remixing-based models allocate only 26% to embeddings (operating on compact base), demonstrating fundamentally different trade-offs: selection prioritizes embedding capacity, while remixing prioritizes routing sophistication.

6 Conclusion

We introduced two approaches for context-aware embedding routing that challenge the assumption that embedding quality scales monotonically with dimensionality. Our selection-based method activates dimensions from a larger pool, while our remixing-based method learns to reweight a compact base embedding with repetition. Both employ mixture-of-experts architectures with soft, context-dependent routing and auxiliary losses encouraging expert specialization.

Across five language modeling benchmarks, activating only 64 dimensions through learned routing consistently matches or outperforms dense 512-dimensional baselines, despite using 8× fewer active dimensions. The remixing approach achieves this with 82.5% fewer parameters (12.32M vs 70.46M), delivering a 3.6× improvement in parameter efficiency on Penn TreeBank. Our analysis reveals that experts develop interpretable specialization patterns and learn genuine contextual disambiguation, routing ambiguous words like "bank" to different experts based on semantic context.

These findings suggest that the structure of dimensional allocation—specifically, learned context-dependent routing—provides stronger inductive bias than raw dimensional budget. While our experiments focus on relatively small models, the results open promising directions including hierarchical expert organization, dynamic expert allocation, and extension to multilingual and multimodal settings. As language models continue to grow, context-adaptive embedding routing offers a path toward parameter-efficient models that maintain or exceed the performance of their larger counterparts while reducing environmental impact.

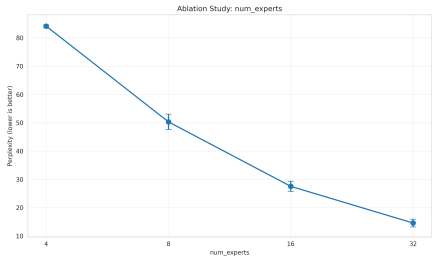

Figure 10. Ablation study on number of experts. Perplexity decreases with more experts up to 16, then shows diminishing returns. Error bars indicate standard deviation across 3 seeds.

Table 3. Parameter count comparison. Remixing achieves massive parameter reduction while maintaining performance.

Model	Total (M)	Embedding (M)	Non-Emb (M)
baseline_512	70.46	25.73	44.73
baseline_256	30.54	12.87	17.67
baseline_64	6.79	3.22	3.57
moe_selection	33.26	28.60	4.66
moe_remixing	12.32	3.22	9.10

References

Andreas, J., Rohrbach, M., Darrell, T., and Klein, D. Neural module networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 39–48, 2016.

Baevski, A. and Auli, M. Adaptive input representations for neural language modeling. *arXiv preprint arXiv:1809.10853*, 2018.

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Bengio, Y., Ducharme, R., Vincent, P., and Jauvin, C. A

neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Yu, A. W., Firat, O., et al. Glam: Efficient scaling of language models with mixture-of-experts. *International Conference on Machine Learning*, pp. 5547–5569, 2022.

Elbayad, M., Gu, J., Grave, E., and Auli, M. Depth-adaptive transformer. *arXiv preprint arXiv:1910.10073*, 2019.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Frankle, J. and Carbin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.

Gao, J., He, D., Tan, X., Qin, T., Wang, L., and Liu, T.-Y. Representation degeneration problem in training natural language generation models. *arXiv preprint arXiv:1907.12009*, 2019.

Grave, É., Joulin, A., Cissé, M., Jégou, H., et al. Efficient softmax approximation for gpus. *International Conference on Machine Learning*, pp. 1302–1310, 2017.

Graves, A. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

Jaszczur, S., Chowdhery, A., Mohiuddin, A., Guseynov, L., Michalewski, H., Gajewski, J., and Kanade, A. Sparse is enough in scaling transformers. *Advances in Neural Information Processing Systems*, 34:9895–9907, 2021.

Jordan, M. I. and Jacobs, R. A. Hierarchical mixtures of experts and the em algorithm. *Neural Computation*, 6(2): 181–214, 1994.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Kitaev, N., Kaiser, Ł., and Levskaya, A. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Lample, G. and Conneau, A. Cross-lingual language model pretraining. *Advances in Neural Information Processing Systems*, 32, 2019.

Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. Large memory layers with product keys. *Advances in Neural Information Processing Systems*, 32, 2019.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2019.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

Lewis, M., Bhosale, S., Dettmers, T., Goyal, N., and Zettlemoyer, L. Base layers: Simplifying training of large, sparse models. *International Conference on Machine Learning*, pp. 6265–6274, 2021.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Mahoney, M. Large text compression benchmark. http://www.mattmahoney.net/dc/text.html, 2011.

Marcus, M. P., Marcinkiewicz, M. A., and Santorini, B. Building a large annotated corpus of english: The penn treebank. *Computational Linguistics*, 19(2):313–330, 1993.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 2013.

Paperno, D., Kruszewski, G., Lazaridou, A., Pham, Q. N., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1525–1534, 2016.

Schuster, T., Fisch, A., Gupta, J., Dehghani, M., Bahri, D., Tran, V. Q., Tay, Y., and Metzler, D. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 34:17456–17472, 2021.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Shen, S., Dong, Z., Ye, J., Ma, L., Yao, Z., Gholami, A., Mahoney, M. W., and Keutzer, K. Q8bert: Quantized 8bit bert. *arXiv preprint arXiv:1910.06188*, 2020.

Smith, L. N. and Topin, N. Super-convergence: Very fast training of neural networks using large learning rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, volume 11006, pp. 369–386. SPIE, 2019.

Sukhbaatar, S., Grave, E., Bojanowski, P., and Joulin, A. Adaptive attention span in transformers. *arXiv preprint arXiv:1905.07799*, 2019.

Svenstrup, D. T., Hansen, J., and Winther, O. Hash embeddings for efficient word representations. *Advances in Neural Information Processing Systems*, 30, 2017.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Xin, J., Tang, R., Lee, J., Yu, Y., and Lin, J. Deebert: Dynamic early exiting for accelerating bert inference. *arXiv preprint arXiv:2004.12993*, 2020.

Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33:17283–17297, 2020.

Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A. M., Chen, Z., Le, Q. V., and Laudon, J. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35:7103–7114, 2022.

Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N., and Fedus, W. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.